```
builder.Services.AddSingleton<DataContext>();
```

Then we can call the `RowCounts` handler and observe the results in figure 9.9. We can see that every instance has returned the same value, indicating that the same instance of `DataContext` is used in every request, both when injected directly into the endpoint handler and when referenced transitively by `Repository`.

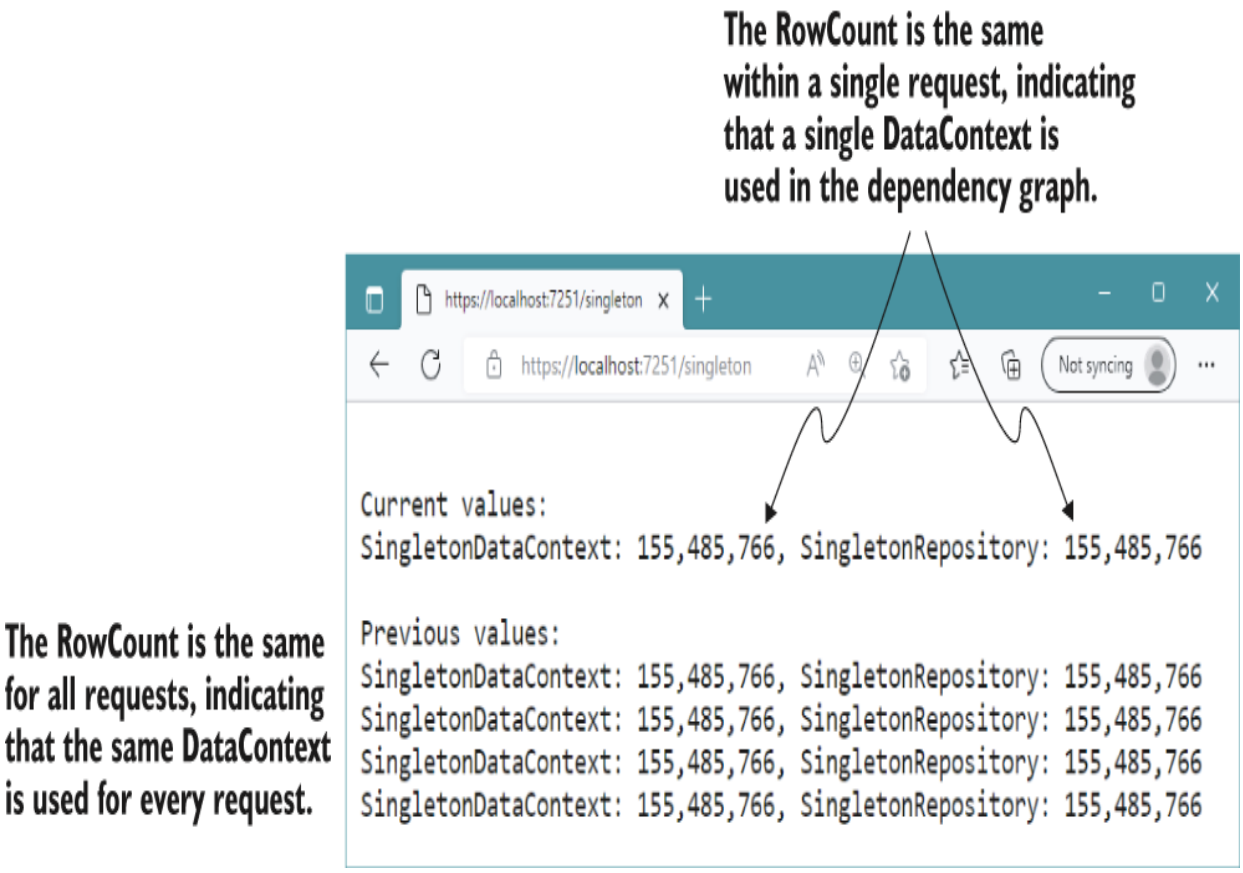


Figure 9.9 Any service registered as a singleton always returns the same instance. Consequently, all the calls to the `RowCounts` handler return the same value, both within a request and between requests.

Singletons are convenient for objects that need to be shared or that are immutable and expensive to create. A caching service should be a singleton,